

Security with TFDSecurityOptions

The built in security features in Delphi 12 will assist you as a developer to add some security features in your database applications.

They will not replace common sense.

Start a new Delphi project and add a Datamodule (File -> New -> Datamodule).

On the datamodule drop a TFDConnection, set it up to point to any database you want to work with. I've set mine up to point to dbdemos.gdb, an Interbase database with the *sysdba* user and password *masterkey*.

Add also a TFDQuery component with the SQL

select * from biolife

Or whatever is meaningful in your database.

By standard properties the UpdateOptions.RequestLive is true, meaning that the dataset is not read only.

On the main forms pas-file, add the datamodule to the uses clause and on the form add a TDataSource and a TDBGrid. Hooking this up with the properties will let the user open up the query and view the data.

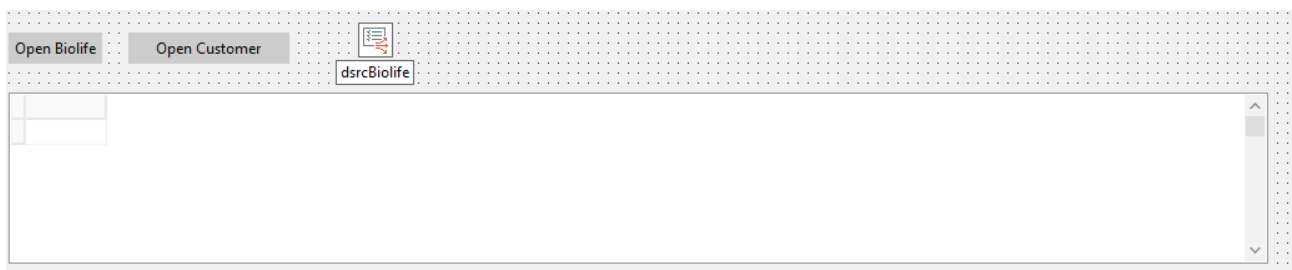
SPECIES_NO	CATEGORY	COMMON_NAME	SPECIES_NAME	LENGTH_CM	LENGTH_IN	NOTES	GRAPHIC
90030	Snapper	Red Emperor	Lutjanus sebae	60	10472440945	(MEMO)	(BLOB)
90050	Wrasse	Giant Maori Wrasse	Cheilinus undulatus	229	4803149606	(MEMO)	(BLOB)
90070	Angelfish	Blue Angelfish	Pomacanthus nauarchus	30	0236220472	(MEMO)	(BLOB)
90080	Cod	Lunartail Rockcod	Variola louti	80	16062992126	(MEMO)	(BLOB)
90090	Scorpio	dsrcBiolife refish	Pterois volitans	38	16299212598	(MEMO)	(BLOB)

Here I just set the queries active property to true at design time

AllowSQL Change

Now set the query.SecurityOptions.AllowSQLChange := false.

On the main form add 2 buttons



With assigned code to “Open Biolife”

```
dmSecurity.fdqryBiolife.Close;
dmSecurity.fdqryBiolife.Open;
```

And on the “Open Customer” button

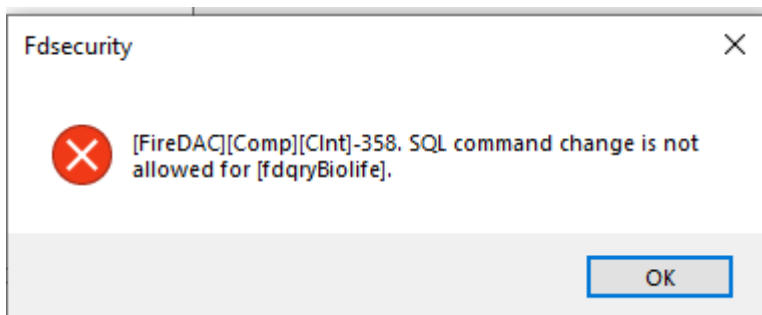
```
dmSecurity.fdqryBiolife.SQL.Clear;
dmSecurity.fdqryBiolife.SQL.Text := 'select * from customer';
dmSecurity.fdqryBiolife.Close;
dmSecurity.fdqryBiolife.open;
```

Which normally would clear the SQL, Assign a new SQL to the SQL.Text property and display the Customer data, in the grid.

Open Biolife works by opening the Query and displaying the data

Open Biolife		Open Customer					
SPECIES_NO	CATEGORY	COMMON_NAME	SPECIES_NAME	LENGTH_CM	LENGTH_IN	NOTES	GRAPHIC
90030	Snapper	Red Emperor	Lutjanus sebae	60	10472440945	(MEMO)	(BLOB)
90050	Wrasse	Giant Maori Wrasse	Cheilinus undulatus	229	4803149606	(MEMO)	(BLOB)
90070	Angelfish	Blue Angelfish	Pomacanthus nauarchus	30	0236220472	(MEMO)	(BLOB)
90080	Cod	Lunartail Rockcod	Variola louti	80	16062992126	(MEMO)	(BLOB)
90090	Scorpionfish	Firefish	Pterois volitans	38	16299212598	(MEMO)	(BLOB)

But the Open Customer button will raise an exception



The SQL is not changed and remains to be Select * from Biolife

AllowedCommandKinds

Now add another TFDQuery to your datamodule, and a Button, memo, TDataSource and a TDBGrid to the main form. Link the TDBGrid to the TDataSource and the TDataSource to the new TFDQuery.

On the TFDQuery set the SecurityOptions.AllowedCommandKinds.skDelete := False

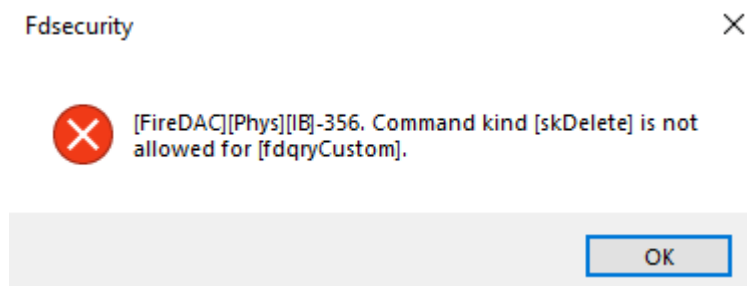
On the buttons click add the code:

```
dmSecurity.fdqryCustom.SQL.Clear;
dmSecurity.fdqryCustom.SQL.Assign(mmoSQL.Lines);
dmSecurity.fdqryCustom.Close;
dmSecurity.fdqryCustom.Open;
```

Which obviously will let the end user send any SQL they want to

SELECT * FROM CUSTOMER will do just that

DELETE FROM CUSTOMER WHERE CUSTNO = 1221 will however raise an exception:



Note1

Setting the TFDQuery. UpdateOptions.RequestLive := true, which is the default behaviour will NOT hinder the user in using Ctrl-Delete on the grid, nor will it hinder you writing code like:

```
dmSecurity.fdqryCustom.delete;
```

Note!

The security options can be set on the individual TFDQuery components like I have done here, or on the TFDConnection component in the standard waterfall model.